

Threads on Your Phone

Nothing is downloaded and nothing is sent anywhere. Every number below is measured on the device you are holding. Keep the screen on and the browser in front, or the numbers are meaningless. **Run each section at least twice.** A phone is a real machine but a noisy instrument — we measured the same device 1.3× apart between two sessions, and charging it made things noisier, not steadier. Trust what repeats: where the curve bends, and which worker made things worse. Not the milliseconds.

1. What did you bring?

hardwareConcurrency : 4 threads
deviceMemory : not reported
platform : Win32
user agent : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36 Edg/140.0.0.0

hardwareConcurrency is the number of threads the browser will let you run. Whether that is also the number of cores, and whether those cores are the same as each other, is what the rest of this page is for.

2. Does one more thread do one more thread's work?

The same fixed amount of arithmetic, split evenly across 1, 2, 3, ... workers. If threads were free, the bar would double every time you double the workers.

Run scaling sweep

workers	time	speedup	efficiency
1	848 ms	1.00×	100%
2	480 ms	1.77×	88%
3	353 ms	2.40×	80%
4	314 ms	2.70×	68%
5	274 ms	3.10×	62%
6	275 ms	3.09×	51%
7	290 ms	2.92×	42%
8	273 ms	3.11×	39%

Perfect scaling would read 8.00× and 100% on the last row. Write down the worker count where the speedup column stops climbing, and compare it with your 4 reported threads.

3. The same loop, handed out two different ways

4096 iterations where iteration i costs about i — the last one is thousands of times more expensive than the first. Same total work both times. **Block** cuts the range into contiguous slabs up front. **Dynamic** makes each worker come back for a small chunk whenever it runs out.

Run both schedules

schedule	makespan	busiest	idlest	imbalance	vs block
block (static)	3786 ms	3289	560	1.59	1.00×
dynamic, chunk 64	2148 ms	2025	1987	1.01	1.76×
dynamic, chunk 8	1959 ms	1669	1545	1.03	1.93×

4 workers. Imbalance is the busiest worker divided by the average; 1.00 means everyone worked the same amount. Nobody did less work under dynamic — they just stopped waiting on each other.

4. Where the arrows say no

Ten tasks with prerequisites. Adding workers stops helping once the makespan reaches the longest chain, no matter how many workers you have.

Run task graph

work $T_1 = 5200$ ms | span $T_\infty = 2750$ ms | most parallelism worth buying = 1.9 workers

workers	makespan	lower bound	speedup	efficiency
1	5252 ms	5200 ms	0.99×	99%
2	3465 ms	2750 ms	1.50×	75%
3	2777 ms	2750 ms	1.87×	62%
4	2772 ms	2750 ms	1.88×	47%
6	2782 ms	2750 ms	1.87×	31%
8	2824 ms	2750 ms	1.84×	23%